CSC 421 - Machine Learning 1

Learning Probabilistic Models

Instructor: Shengyao Lu

Readings

Chapter 20 LEARNING PROBABLISTIC MODELS

- Basic: Sections **20.1, 20.2.1, 20.2.2**, and **Summary**
- Expected: Same as Basic plus **20.3, 20.3.1**
- Advanced: All the chapter including bibliographical and historical notes

Be sure you are familiar with the pseudocode. You may be tested on your understanding of it in the final.

Recall: Agents perceive their environment through sensors and act upon that environment through actuators.

When the agent is a computer, we call it **machine learning**: a computer **observes** some data, **builds a model** based on the data, and uses the model as both a hypothesis about the world and a piece of software that can **solve problems**.

Why we need machine learning?

1. The designers cannot anticipate all possible future situations.
2. Second, sometimes the designers have no idea how to program a solution themselves.

1. Types of learning

- **Supervised learning**: correct answers for each example
  - requires “teacher” (label)
  - e.g., classification, regression
- **Unsupervised learning**: requires no teacher (unlabeled), but harder
  - looking for interesting **patterns** in examples
  - e.g., k-means (clustering), PCA (reduces the number of dimensions in large datasets)
  - **Self-supervised learning**: learning from unlabeled data in a supervised-learning manner, by generating labels from the data itself, often through predicting part of the data from other parts.

        - e.g., Word2Vec, BERT, GPT, Contrastive Learning, Auto-Encoder, GAN, Diffusion models

- **Semisupervised learning**: between supervised & unsupervised learning
    - improve performance in one of these two by utilizing information associated with the other
- **Reinforcement learning**: occasional rewards
    - tries to maximize the rewards

Other types:

- **Transfer learning**: learning a new task through the transfer of targets from a related source task that has already been learned
- **Ensemble learning**: multiple learners are trained to solve the same problem
- **Federated learning**: many clients collaboratively train a model
- **Explanation-based Learning**: learning in knowledge

2. Statistical learning

- When the structure of a Bayesian network and its conditional probability tables (CPTs) are given, we can perform probabilistic reasoning to answer queries.
- However, when the parameters or the structure of the network are unknown, we need statistical learning methods to estimate the CPTs and possibly learn the network structure **from data**.
- Estimated the CPTs and learned the network structure can be viewed as **hypotheses**.
- The observed data can be viewed as **evidence**.

We often learn two types of things:

1. **Parameter learning**: finding the numerical parameters for a probability model whose structure is fixed.
    - e.g. Given BN, learn CPTs with ML approximation.
2. **Structure learning**: finding the structure of a probabilistic model (e.g., Bayes net) from data by fitting the parameters

We will introduce three parameter learning methods that apply across different learning paradigms.

General Concepts of Supervised Learning

Agents learn: $f : x \rightarrow y$ from training data, and test on testing data.

- X: features
- Y: labels
- dataset: $[train, (valid), test]$
    - each data sample: (x, y)

2.1 Bayesian learning

- H is the hypothesis variable with values $\ldots h_i \ldots$, prior $P(H)$.
  - The j-th observation d_j gives the outcome of random variable $D_j \in D$
  - training data $\mathbf{d} = d_1, \ldots, d_N$, all the data points are observable (no hidden variables).
- **Bayesian learning**: calculate posterior $\mathbf{P}(h_i|\mathbf{d})$ for each h_i in H.
  - i.e., given the evidence $\mathbf{P}(\mathbf{d})$, which model hypothesis is more possible?
    - In other words, if h_i is true, how likely it will meet with the evidence $\mathbf{d}$?
  - Mathematically, we can calculate the probability of each hypothesis by weighting the likelihood by the prior by Bayes' rule:

$$\mathbf{P}(h_i|\mathbf{d}) = \alpha \mathbf{P}(\mathbf{d}|h_i)\mathbf{P}(h_i)$$

  - $\mathbf{P}(\mathbf{d} \mid h_i)$ is called likelihood, how well hypothesis explains the data.
  - $\mathbf{P}(h_i)$ is prior, our preference over hypotheses before seeing data, can penalize overly complex hypotheses.

$$Posterior = \frac{likelihood \times prior}{Evidence}$$

- Predictions about an unknown quantity Y use a likelihood-weighted average over the hypotheses

$$\mathbf{P}(Y|\mathbf{d}) = \sum_i \mathbf{P}(Y|\mathbf{d}, h_i)\mathbf{P}(h_i|\mathbf{d}) = \sum_i \mathbf{P}(Y|h_i)\mathbf{P}(h_i|\mathbf{d})$$

  where we assume that predictions are weighted averages over the predictions of the individual hypotheses. The key terms are the **hypothesis** prior $P(h_i)$ and the **likelihood** of the data under each hypothesis $P(\mathbf{d}|h_i)$.

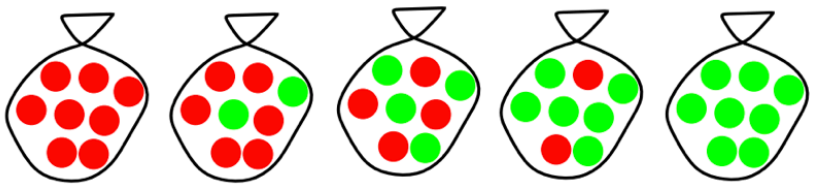
- Assuming that the observations are i.i.d. (Independent and Identically Distributed) we can calculate easily the likelihood like we did before, and the priors for the hypotheses are given:

$$P(\mathbf{d}|h_i) = \prod_j P(d_j \mid h_i)$$

- Example

Suppose there are five kinds of bags of candies with prior distribution

10% are h_1: 100% cherry candies
20% are h_2: 75% cherry candies + 25% lime candies
40% are h_3: 50% cherry candies + 50% lime candies
20% are h_4: 25% cherry candies + 75% lime candies
10% are h_5: 100% lime candies



Then we observe candies drawn from some bag:

What kind of bag is it? What flavour will the next candy be?

2.2 MAP approximation and ML approximation

- Summing over the hypothesis space is often intractable
  - e.g., in continuous hypothesis space, sum means integration, which usually costly and may not have closed form solution.
  - If we can turn it to a optimization problem, it would be much easier.
- **Maximum a posteriori (MAP) learning**: Choose h_{MAP} that maximizes $P(h_i \mid \mathbf{d})$, i.e.,

$$h_{\text{MAP}} = \arg\max_i \; P(\mathbf{d} \mid h_i)\, P(h_i) \quad \text{or} \quad h_{\text{MAP}} = \arg\max_i \; \log P(\mathbf{d} \mid h_i) + \log P(h_i)$$

  - Log terms can be viewed as the (negative of):
    - **bits to encode data given hypothesis + bits to encode hypothesis**
    - For example, no bits are required for certain hypotheses such as h_5, $\log_2 1 = 0$
    - This is the basic idea of **minimum description length (MDL) learning**.
  - Then $P(Y \mid \mathbf{d}) \approx P(Y \mid h_{MAP}) \therefore$ no needs to sum.
- **Maximum-likelihood (ML) learning**: choose h_{ML} maximizing $P(\mathbf{d} \mid h_i)$, i.e.,

$$h_{ML} = \arg\max_i \; P(\mathbf{d} \mid h_i) \quad \text{or} \quad h_{ML} = \arg\max_i \; \log P(\mathbf{d} \mid h_i)$$

- For large datasets, prior becomes irrelevant.
- identical to MAP for uniform prior $P(\mathbf{d} \mid h_i)\, P(h_i) = P(\mathbf{d} \mid h_i)$
- ML is the non-Bayesian statistical learning

2.3 EM algorithm

- When we have **hidden variables** in the problem, we say the data is incomplete.
- There are many scenarios where this is the case. For example, the book describes:
  1. Unsupervised clustering with mixtures of Gaussians
  2. Learning Bayesian networks with hidden variables
  3. Learning HMMs from observations without associated states

In all these cases, a common approach is to use the **Expectation-Maximization (EM) algorithm**.

- EM algorithm is an iterative algorithm in which, at each step, we improve our estimation of the parameters of a probabilistic model.
  - **Pretend** that we know the parameters of the model
    - infer the probability that each data belongs to each compnt.
  - **Refit** the components to the entire data set
    - with each data weighted by the probability that it belongs to
  - the process iterates until **convergence**
- Denote model parameters as θ, evidence (observed data) as X, hidden variables (unobserved data) as Z. The goal is to find the hypothsis that maximize log-likelihood:

$$\theta^* = \arg\max_{\theta} \; \log P(X \mid \theta)$$

1. **E-step** (expectation): take expectation w.r.t. the posterior probability over hidden variables Z given current model.

$$Q(\theta \mid \theta^{(t)}) = \mathbb{E}_{Z|X,\theta^{(t)}} \left[\log \; P(X, Z \mid \theta)\right] = \sum_{Z} P(Z \mid X, \theta^{(t)}) \log P(X, Z \mid \theta)$$

2. **M-step** (maximum likelihood): Maximizes the expected log-likelihood to update parameter estimates using the complete data from the E-step.

$$\theta^{(t+1)} = \arg\max_{\theta} \; Q(\theta \mid \theta^{(t)})$$